Міністерство освіти і науки України
Національний технічний університет
«Дніпровська політехніка

Звіт
Про виконання Лабораторної роботи
З дисципліни «Розробка REST-застосунку на Spring Boot»

Виконав
Студент гр. 124-22-1
Возіянов Денис Олександрович

Перевірив
Доцент САУ
Мінієєв Олександр Сергійович

Дніпро
2026

## Завдання

**1. Мета роботи**

Розробити повноцінний клієнт-серверний застосунок на мові Java, що реалізує обрану бізнес-логіку (наприклад, прототип вашої майбутньої дипломної роботи або сервіс для автоматизації власних задач). Сама логіка може бути будь яка. Жодних вимог. Уявіть що це ваш старт-ап

**2. Технологічний стек**

• Backend: Java 17+, Spring Boot 3.x.
• Database: Будь-яка реляційна БД (PostgreSQL, MySQL, H2) з використанням Spring Data JPA.
• Frontend: Серверна візуалізація можна зробити будь-якою технологією наприклад на базі шаблонізатора Thymeleaf.
• Data Processing: Бібліотека для парсингу (наприклад, Jsoup для HTML або Jackson/Gson для зовнішніх JSON API).

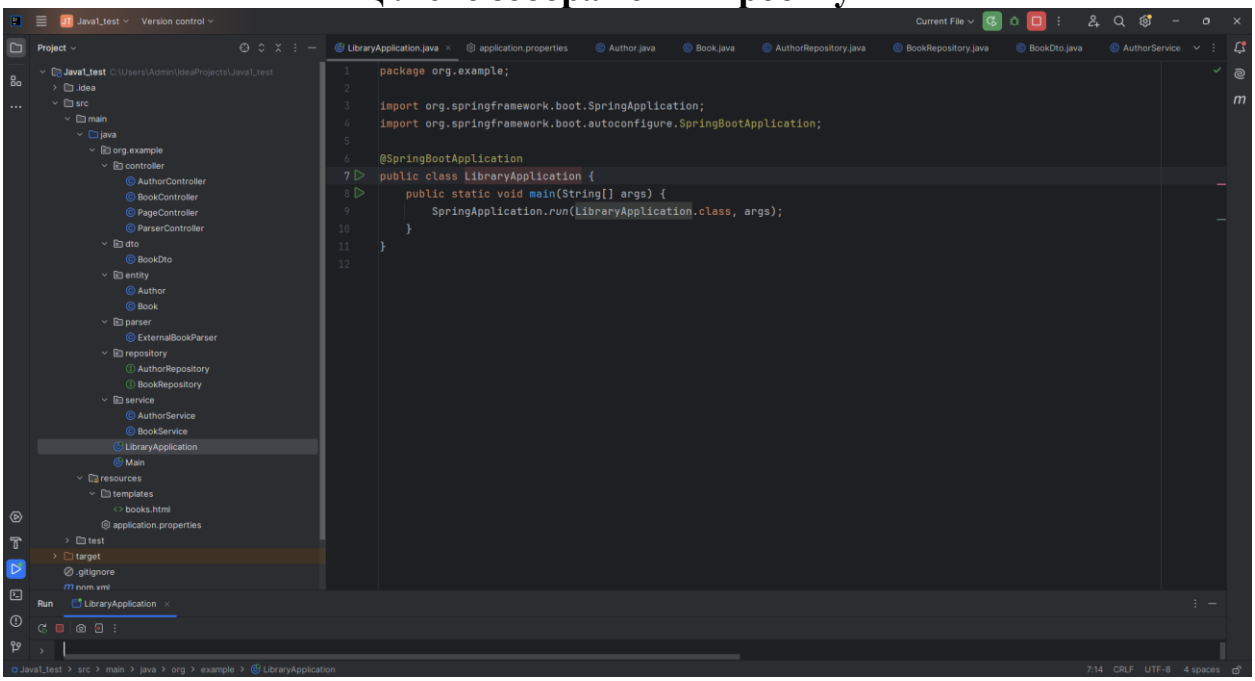
**3. Основні вимоги до функціоналу**

A. Серверна частина (REST API)
• Реалізація контролерів (@RestController або @Controller).
• Підтримка базових CRUD-операцій (Створення, Читання, Оновлення, Видалення) для основних сутностей.
• Використання DTO (Data Transfer Object) для передачі даних між шарами застосунку. (Бажано!)
B. Робота з даними (Database + Parser)
• База даних: Створення мінімум 2-3 пов'язаних таблиць (One-to-Many або Manyto-Many).
• Парсер та request API: У застосунку має бути реалізований модуль, який збирає дані з зовнішнього джерела.

## Виконання цього завдання

**Тема:** Розробка веб-застосунку на Spring Boot для роботи з авторами та книгами: REST-API, база даних та HTML-представлення.

## Цілісне зображення проєкту




## Controller

## AuthorController

```
package org.example.controller;

import lombok.RequiredArgsConstructor;
import org.example.entity.Author;
import org.example.service.AuthorService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/authors")
@RequiredArgsConstructor
public class AuthorController {

    private final AuthorService authorService;

    @GetMapping
    public List<Author> all() {
        return authorService.findAll();
    }

    @PostMapping
    public Author create(@RequestParam("name") String name) {
        return authorService.create(name);
    }
}
```

## BookController

```
package org.example.controller;

import lombok.RequiredArgsConstructor;
import org.example.dto.BookDto;
import org.example.entity.Book;
import org.example.service.BookService;
import org.springframework.web.bind.annotation.*;
```

```java
import java.util.List;

@RestController
@RequestMapping("/api/books")
@RequiredArgsConstructor
public class BookController {

    private final BookService bookService;

    @GetMapping
    public List<Book> all() {
        return bookService.findAll();
    }

    @PostMapping
    public Book create(@RequestBody BookDto dto) {
        return bookService.create(dto);
    }
}
```

## PageController

```java
package org.example.controller;

import lombok.RequiredArgsConstructor;
import org.example.service.BookService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
@RequiredArgsConstructor
public class PageController {

    private final BookService bookService;

    @GetMapping("/books")
    public String booksPage(Model model) {
        model.addAttribute("books", bookService.findAll());
        return "books";
    }
}
```

## ParserController

```java
package org.example.controller;

import lombok.RequiredArgsConstructor;
import org.example.parser.ExternalBookParser;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequiredArgsConstructor
public class ParserController {

    private final ExternalBookParser parser;

    @GetMapping("/api/parse")
    public String parse(@RequestParam String q) throws Exception {
        parser.loadBooks(q);
        return "OK";
```

```
    }
}
```

## dto
## BookDto

```
package org.example.dto;

import lombok.Data;

@Data
public class BookDto {
    private Long id;
    private String title;
    private Long authorId;
}
```

## entity
## Author

```
package org.example.entity;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.*;

import java.util.List;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @JsonIgnore
    @OneToMany(mappedBy = "author")
    private List<Book> books;
}
```

## Book

```
package org.example.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
```

```
    @ManyToOne
    private Author author;
}
```

## Parser
## ExternalBookParser

```
package org.example.parser;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import org.example.entity.Book;
import org.example.repository.BookRepository;
import org.springframework.stereotype.Service;

import java.net.URL;

@Service
@RequiredArgsConstructor
public class ExternalBookParser {

    private final BookRepository bookRepository;

    public void loadBooks(String query) throws Exception {
        String url = "https://www.googleapis.com/books/v1/volumes?q=" +
query;

        ObjectMapper mapper = new ObjectMapper();
        JsonNode root = mapper.readTree(new URL(url));
        JsonNode items = root.get("items");
        if (items == null) return;

        for (JsonNode item : items) {
            String title = item.get("volumeInfo").get("title").asText();

            Book b = new Book();
            b.setTitle(title);
            b.setAuthor(null); // можно не заполнять автора

            bookRepository.save(b);
        }
    }
}
```

## Repository
## Author Repository

```
package org.example.repository;

import org.example.entity.Author;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AuthorRepository extends JpaRepository<Author, Long> {
}
```

## Book Repository

```
package org.example.repository;

import org.example.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

## Service

## AuthorService

```
package org.example.service;

import lombok.RequiredArgsConstructor;
import org.example.entity.Author;
import org.example.repository.AuthorRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class AuthorService {

    private final AuthorRepository authorRepository;

    public List<Author> findAll() {
        return authorRepository.findAll();
    }

    public Author create(String name) {
        Author a = new Author();
        a.setName(name);
        return authorRepository.save(a);
    }
}
```

## BookService

```
package org.example.service;

import lombok.RequiredArgsConstructor;
import org.example.dto.BookDto;
import org.example.entity.Author;
import org.example.entity.Book;
import org.example.repository.AuthorRepository;
import org.example.repository.BookRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class BookService {

    private final BookRepository bookRepository;
    private final AuthorRepository authorRepository;

    public List<Book> findAll() {
        return bookRepository.findAll();
    }

    public Book create(BookDto dto) {
        Author author = authorRepository.findById(dto.getAuthorId())
                .orElseThrow(() -> new RuntimeException("Author not found"));

        Book b = new Book();
        b.setTitle(dto.getTitle());
        b.setAuthor(author);
```

```
        return bookRepository.save(b);
    }
}
```

## LibraryApplication

```
package org.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryApplication.class, args);
    }
}
```

## resources/templates
## books.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Books</title>
</head>
<body>
<h1>Books</h1>
<ul>
    <li th:each="b : ${books}" th:text="${b.title}"></li>
</ul>
</body>
</html>
```

## application.properties

```
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

server.port=8080
```

## pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>library-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <java.version>17</java.version>
        <spring.boot.version>3.2.0</spring.boot.version>
    </properties>
```

```
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-dependencies</artifactId>
                <version>${spring.boot.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>
        <!-- Web -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- JPA -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- H2  -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Thymeleaf -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>

        <!-- Lombok -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

**Опис реалізації**

**Структура проєкту**

Пакет entity:

- Author — сутність автора
- Book — сутність книги

Пакет repository:

- AuthorRepository
- BookRepository

Пакет service:

- AuthorService
- BookService

Пакет controller:

- AuthorController (/api/authors)
- BookController (/api/books)
- PageController (/books — HTML-сторінка)

**Сутність Author**

Клас Author:

поля:

- id: Long — ідентифікатор
- name: String — ПІБ автора
- books: List<Book> — список книг автора

основні анотації:

- @Entity
- @Id, @GeneratedValue
- @OneToMany(mappedBy = "author")
- @JsonIgnore для уникнення рекурсії при серіалізації
- Lombok: @Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor

**Сутність Book**

Клас Book:

поля:

- id: Long — ідентифікатор
- title: String — назва книги
- author: Author — посилання на автора

анотації:

- @Entity
- @ManyToOne
- @JoinColumn(name = "author_id")

REST-контролери

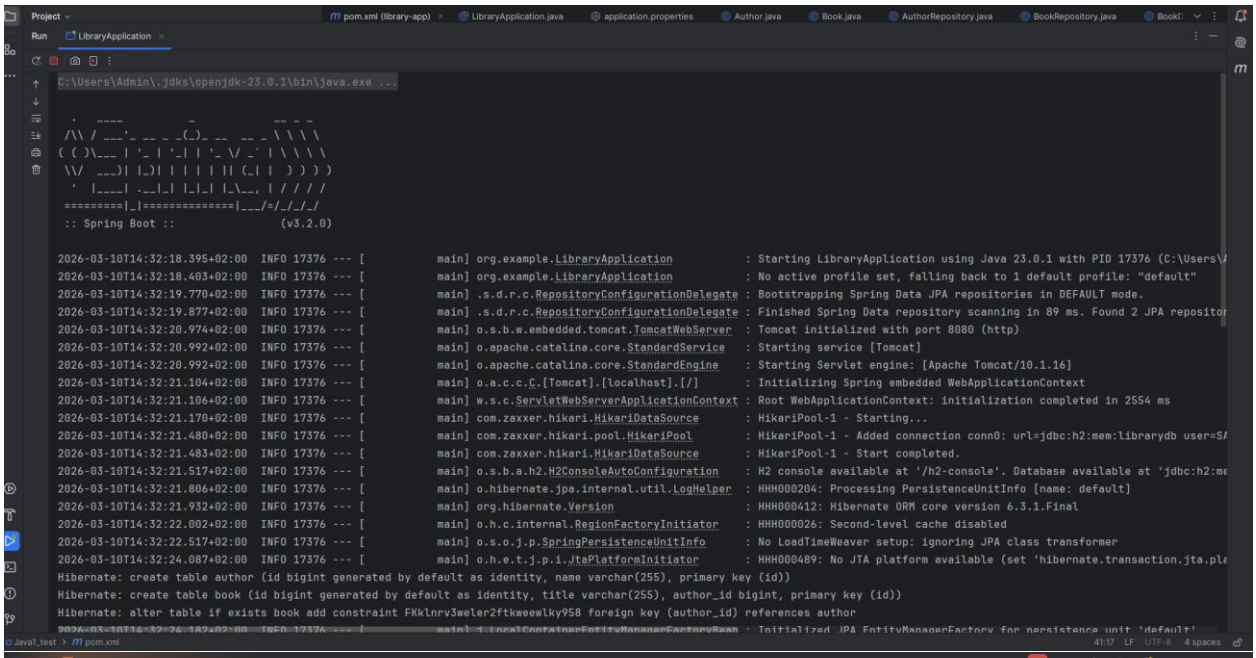
AuthorController (/api/authors):

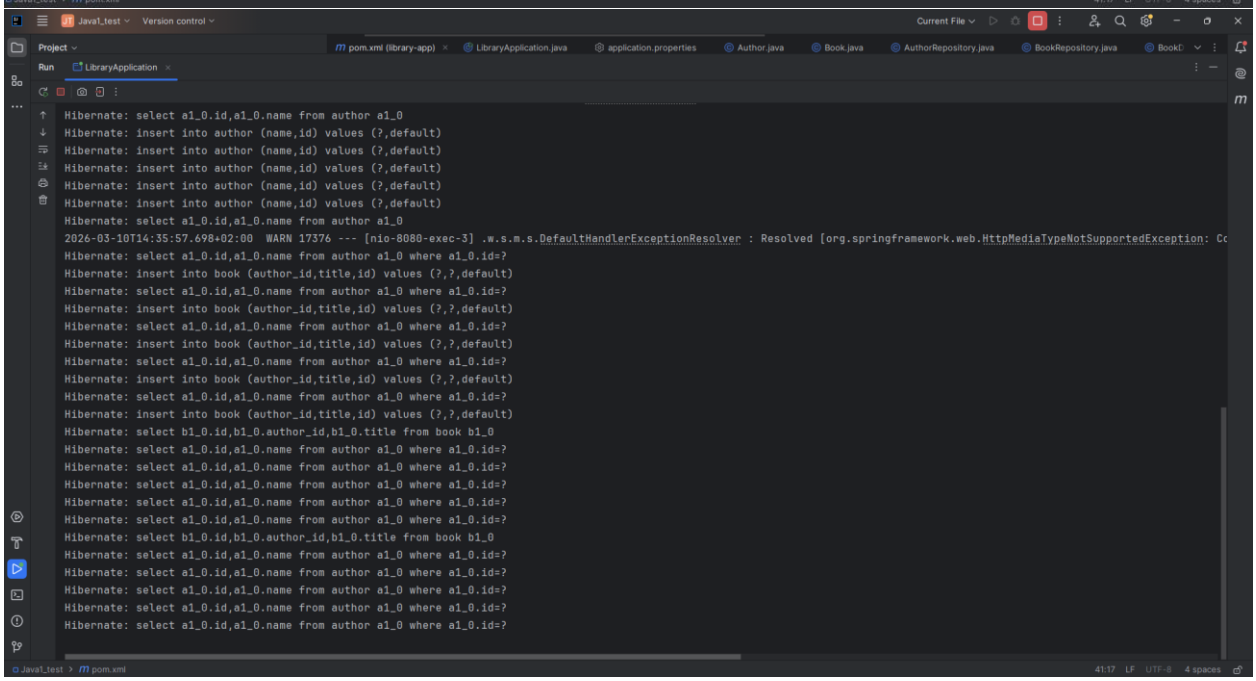
- GET /api/authors — отримати всіх авторів
- POST /api/authors — створити автора
    - параметр: name (form-data / request param)

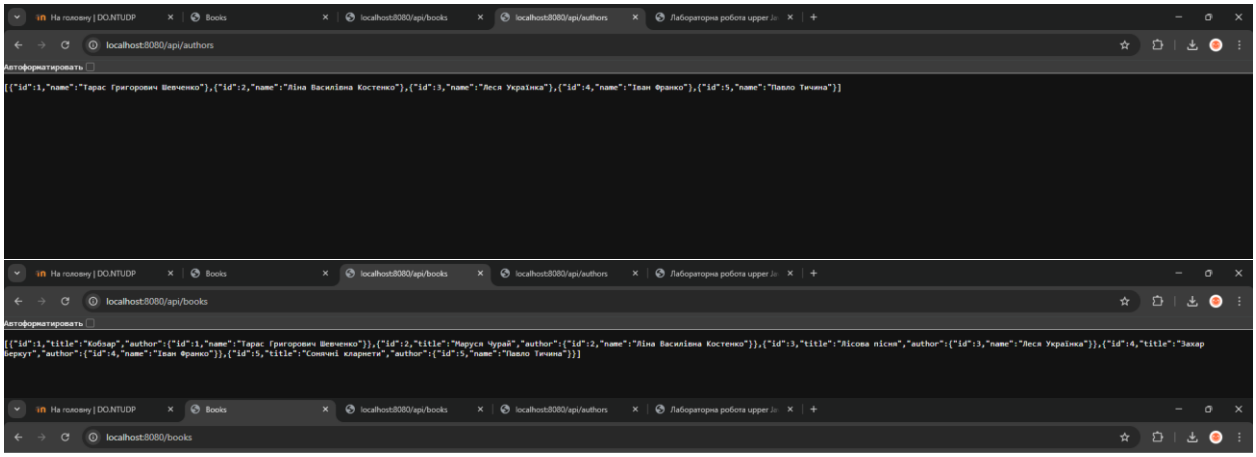
BookController (/api/books):

- GET /api/books — отримати всі книги
- POST /api/books — створити книгу
    - тіло запиту (JSON):
      {
      "title": "Кобзар",
      "authorId": 1
      }

## Тестування

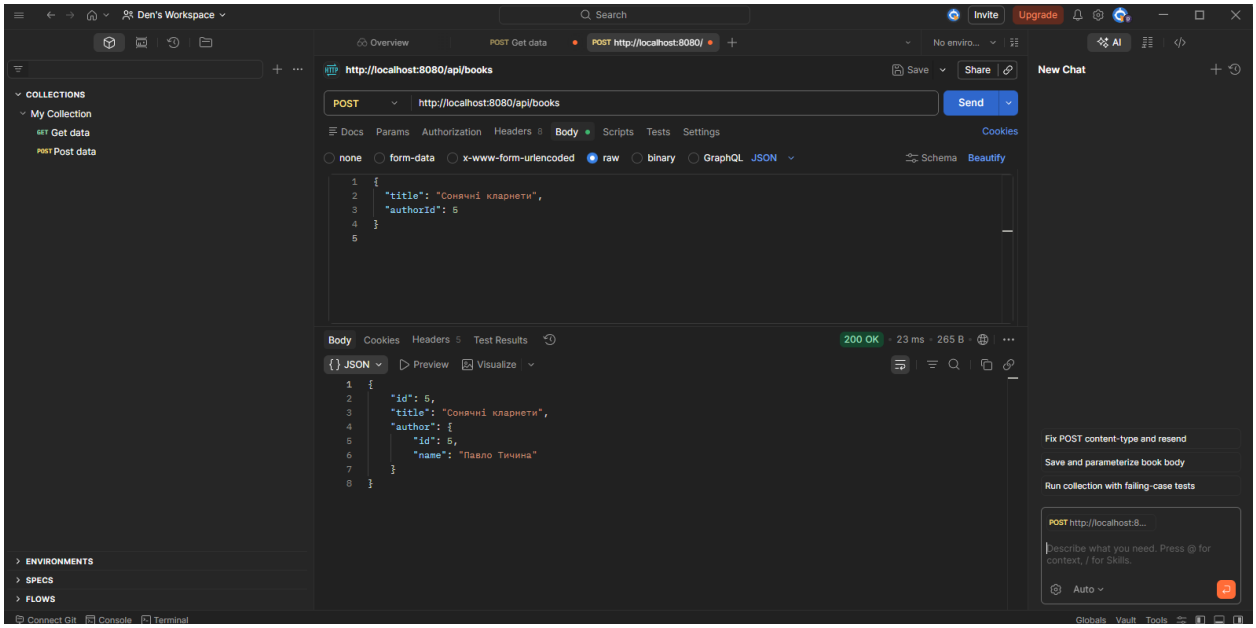





## Тестування API в Postman

- Створено запити:
  - POST /api/authors — додано 5 авторів
  - GET /api/authors — перевірено список авторів
  - POST /api/books — додано 5 книг, кожна прив’язана до автора
  - GET /api/books — перевірено список книг з авторами
- Приклади даних:
  - Автори:
    - Тарас Григорович Шевченко
    - Ліна Василівна Костенко
    - Леся Українка
    - Іван Франко
    - Павло Тичина
  - Книги:
    - «Кобзар» — Шевченко
    - «Маруся Чурай» — Костенко
    - «Лісова пісня» — Леся Українка
    - «Захар Беркут» — Франко
    - «Сонячні кларнети» — Тичина




## Висновки

У ході роботи було:

- розроблено простий веб-застосунок на Spring Boot;
- реалізовано зберігання даних про авторів та книги з використанням JPA;
- налаштовано двосторонній зв’язок між сутностями Author і Book;
- створено REST-інтерфейс для роботи з даними (отримання та створення записів);
- протестовано API за допомогою Postman;
- реалізовано HTML-сторінку для відображення списку книг;
- виправлено типові помилки (500, 415, рекурсія при серіалізації).

**Практичний результат:** Побудовано повноцінний міні-застосунок «Автори та книги», який поєднує базу даних, REST-API та веб-інтерфейс.